CSCI 12532
Computer Architecture and Design
Lecture 07

Ms. Meesha Mervyn
Dept. of Computer Systems Engineering
Faculty of Computing and Technology
University of Kelaniya
meesham@kln.ac.lk

CONTENTS

1. INTRODUCTION TO SPECIALIZED ARCHITECTURES

- Rapid advancements in technology require faster, more efficient computing systems. Specialized architectures address these demands.
- As traditional scaling (Moore's Law) slows, building general-purpose CPUs that handle all workloads perfectly is no longer efficient. Specialized architectures bypass the "Von Neumann bottleneck" by placing logic and targeted memory closer together.

Specialized CPU architectures are tailored processors designed for specific workloads rather than general-purpose computing. By integrating dedicated hardware accelerators or using non-traditional instruction set architectures (ISAs), they maximize efficiency, lower power consumption, and boost performance for targeted tasks like artificial intelligence, edge computing, and signal processing.

2. Moore’s Law

Moore's Law, originally an observation by Gordon E. Moore in 1965, asserts that the number of transistors on a microchip doubles approximately every two years, leading to more powerful and cost-effective computing power.

Moore's Law implies that computers, machines that run on computers, and computing power all become smaller, faster, and cheaper with time as processes become more efficient and components smaller and faster.

According to some, Moore's Law will end sometime in the 2020s. If components continue to shrink, physical limits will be reached during this decade because it's unlikely that transistors smaller than atoms can be printed. There is only 1.5nm of space left to print on, depending on the element.

https://www.youtube.com/watch?v=qmn_x90VMV4

Why Moore’s Law Does Not Apply to AI

1. Modern AI chips feature thousands of cores working simultaneously, enabling massive parallelization of matrix computations. This allows neural networks to process millions of parameters in parallel rather than sequentially, resulting in speed improvements that scale with the number of cores.
2. Specialized Architectures Like GPUs, TPUs, and Custom Accelerators. These purpose-built processors are optimized specifically for AI workloads, with GPUs offering thousands of cores for parallel processing, TPUs designed for tensor operations in neural networks, and custom ASICs like Nvidia’s H100 achieving up to 1000x better performance-per-watt compared to general-purpose CPUs for AI tasks.

3. Algorithmic innovations like transformer architecture, mixture-of-experts, and quantization techniques have enabled efficiency gains independent of hardware. Systems improvements in distributed training, memory management, and optimization techniques have reduced the computational requirements for state-of-the-art models by orders of magnitude compared to naive implementations.

3. Control Unit in Computer Architecture

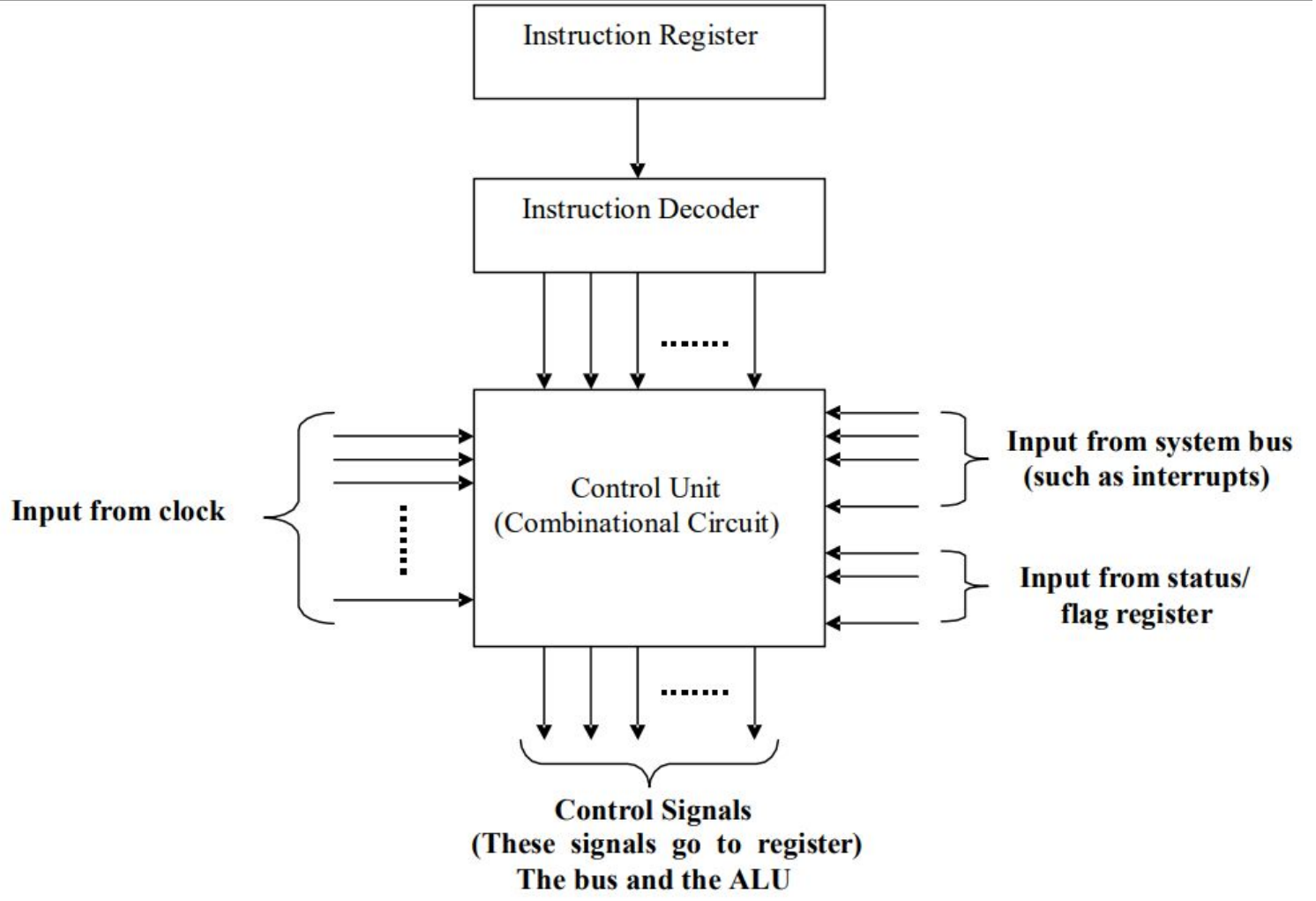
The control unit is a crucial component in a computer's central processing unit (CPU) responsible for coordinating and directing the execution of instructions. It interprets and decodes instructions fetched from memory, controls the flow of data between different parts of the CPU, and manages the overall operation of the processor.

The control unit acts as the brain of the CPU because it controls and manages the execution of instructions. It determines the sequence of operations, directs the flow of data, and ensures proper coordination among different components. Without the control unit, the CPU would be unable to execute instructions or perform any meaningful tasks.

Types of Control Units

1. Hardwired Control

- The hardwired control consists of a combinational circuit that outputs desired controls for decoding and encoding functions. The instruction that is loaded in the IR is decoded by the instruction decoder. If the IR is an 8-bit register, then the instruction decoder generates 28 (256) lines.
- The major goal of implementing the hardwired control is to minimize the cost of the circuit and to achieve greater efficiency in the operation speed.


Instruction Register
Instruction Decoder
Control Unit
(Combinational Circuit)
Input from clock
Input from system bus
(such as interrupts)
Input from status/
flag register
Control Signals
(These signals go to register)
The bus and the ALU

A Hardwired Control unit is composed of two decoders, a sequence counter, and logic gates.
The instruction register (IR) stores instructions fetched from the memory unit.
The instruction register comprises the operation code, the I bit, and bits 0 through 11.
A 3 x 8 decoder is used to encode the operation code in bits 12 through 14.
The outputs of the decoder are represented by the letters D0 through D7.
The operation code of bit 15 is transferred to a flip-flop denoted by the symbol I.
The control logic gates are programmed with operation codes from bits 0 to 11.
The sequence counter (or SC) has the ability to count from 0 to 15 in binary.

Benefits of Using a Hardwired Control Unit

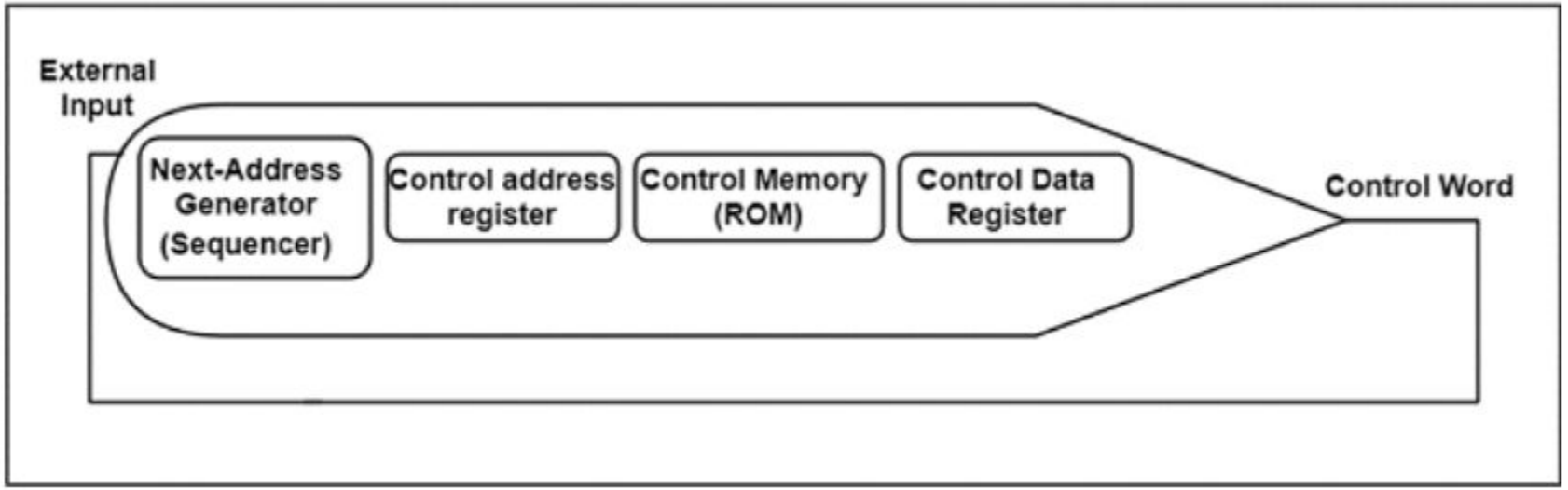
- Because of the use of combinational circuits to generate signals, Hardwired Control Unit is fast.
- It depends on number of gates, how much delay can occur in generation of control signals.
- It can be optimized to produce the fast mode of operation.
- Faster than micro- programmed control unit.
- It does not require control memory.

Limitations of Hardwired Control Unit

- The complexity of the design increases as we require more control signals to be generated (need of more encoders & decoders)
- Modifications in the control signals are very difficult because it requires rearranging of wires in the hardware circuit.
- Adding a new feature is difficult & complex.
- Difficult to test & correct mistakes in the original design.
- It is Expensive.

2. Microprogrammed Control Unit

- A control unit whose binary control values are saved as words in memory is called a microprogrammed control unit.
- A controller results in the instructions to be implemented by constructing a definite collection of signals at each system clock beat. Each of these output signals generates one micro-operation including register transfer. Thus, the sets of control signals are generated definite micro-operations that can be saved in the memory.
- Each bit that forms the microinstruction is linked to one control signal. When the bit is set, the control signal is active. When it is cleared the control signal turns inactive. These microinstructions in a sequence can be saved in the internal ’control’ memory. The control unit of a microprogram-controlled computer is a computer inside a computer.



- The control memory address register specifies the address of the microinstruction, and the control data register holds the microinstruction read from memory.
- The microinstruction contains a control word that specifies one or more microoperations for the data processor. Once these operations are executed, the control must determine the next address.
- The location of the next microinstruction may be the one next in sequence, or it may be located somewhere else in the control memory.

It can execute any instruction. The CPU should divide it down into a set of sequential operations. This set of operations are called microinstruction. The sequential micro-operations need the control signals to execute.

Control signals saved in the ROM are created to execute the instructions on the data direction. These control signals can control the micro-operations concerned with a microinstruction that is to be performed at any time step.

The address of the microinstruction is executed next is generated.

The previous 2 steps are copied until all the microinstructions associated with the instruction in the set are executed.

Advantages of Microprogrammed Control Unit

- It can more systematic design of the control unit.
- It is simpler to debug and change.
- It can retain the underlying structure of the control function.
- It can make the design of the control unit much simpler. Hence, it is inexpensive and less error-prone.
- It can orderly and systematic design process.
- It is used to control functions implemented in software and not hardware.
- It is more flexible.
- It is used to complex function is carried out easily.

Disadvantages of Microprogrammed Control Unit

- Slower Execution: Microprogrammed control units are generally slower than hardwired control units because each instruction requires multiple microinstructions to execute.
- Memory Overhead: Requires additional memory to store the microprogram, which increases the cost and complexity.
- Complex Microprogramming: Writing and debugging microprograms can be complex and time-consuming.
- Less Efficient for Simple Instructions: For simple instructions, the overhead of microprogramming may reduce efficiency compared to hardwired control.
- Additional Fetch Cycles: Each microinstruction needs to be fetched from control memory, which adds to the execution time.

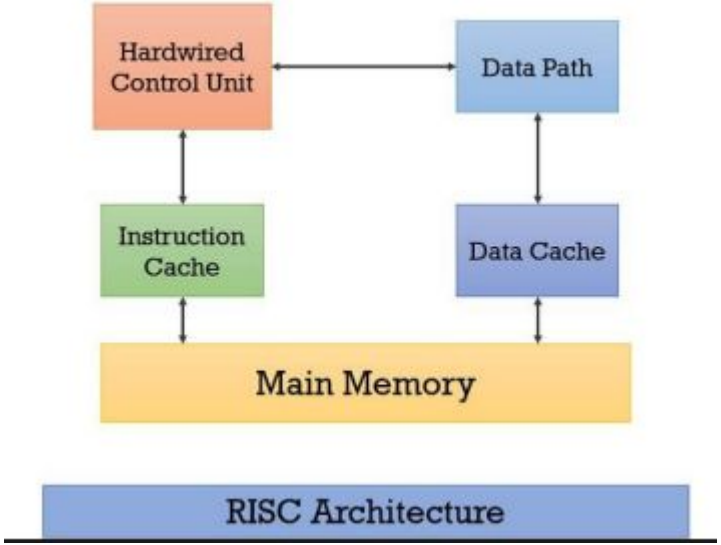
Comparison

Hardwired Control	Microprogrammed Control
Technology is circuit based.	Technology is software based.
It is implemented through flip-flops, gates, decoders etc.	Microinstructions generate signals to control the execution of instructions.
Fixed instruction format.	Variable instruction format (16-64 bits per instruction).
Instructions are register based.	Instructions are not register based.
ROM is not used.	ROM is used.
It is used in RISC.	It is used in CISC.
Faster decoding.	Slower decoding.
Difficult to modify.	Easily modified.
Chip area is less.	Chip area is large.

4. RISC MACHINE COMPONENTS

A RISC machine consists of several crucial components that work together to
execute instructions and process data efficiently.
The main components includes

- Hardwired Control Unit
- Instruction Cache
- Data Cache
- Data Path
- Main Memory.



Hardwired Control Unit:

- Coordinates CPU operations.
- Uses a hardwired design with combinational logic circuits to generate control signals.

Data Path:

- Performs arithmetic and logical operations.
- Includes the ALU (Arithmetic Logic Unit) for calculations.
- Contains registers for temporary data storage.

Instruction Cache:

- Small, high-speed memory for frequently used instructions.
- Reduces fetch time and improves performance.

Data Cache:

- High-speed memory for frequently accessed data.
- Reduces data access time by storing data closer to the CPU.

Main Memory:

- Primary storage for both instructions and data.
- Larger in size but slower than caches.

4. RISC INSTRUCTION SET

Designed to be simple and streamlined with a limited number of instructions.

Focuses on basic operations like arithmetic, logical, and data movement.

Instructions are typically executed in a single clock cycle for fast performance.

Characteristics of RISC Instruction Set

Simple and streamlined

- Provides a small set of basic instructions instead of a large variety of specialized ones.
- Instructions can be combined to perform complex tasks.

Basic operations

- Includes fundamental operations like addition, comparison, data movement, and logical functions.
- Prioritizes essential operations for efficiency.

Single clock cycle execution:

- Each instruction completes within one clock cycle.
- Ensures fast and efficient execution.

Arithmetic Operation

Mnemonic	Instruction	Type	Description
`ADD rd, rs1, rs2`	Add	R	`rd ← rs1 + rs2`
`SUB rd, rs1, rs2`	Subtract	R	`rd ← rs1 - rs2`
`ADDI rd, rs1, imm12`	Add immediate	I	`rd ← rs1 + imm12`
`SLT rd, rs1, rs2`	Set less than	R	`rd ← rs1 < rs2 ? 1 : 0`
`SLTI rd, rs1, imm12`	Set less than immediate	I	`rd ← rs1 < imm12 ? 1 : 0`
`SLTU rd, rs1, rs2`	Set less than unsigned	R	`rd ← rs1 < rs2 ? 1 : 0`
`SLTIU rd, rs1, imm12`	Set less than immediate unsigned	I	`rd ← rs1 < imm12 ? 1 : 0`
`LUI rd, imm20`	Load upper immediate	U	`rd ← imm20 << 12`
`AUIP rd, imm20`	Add upper immediate to PC	U	`rd ← PC + imm20 << 12`

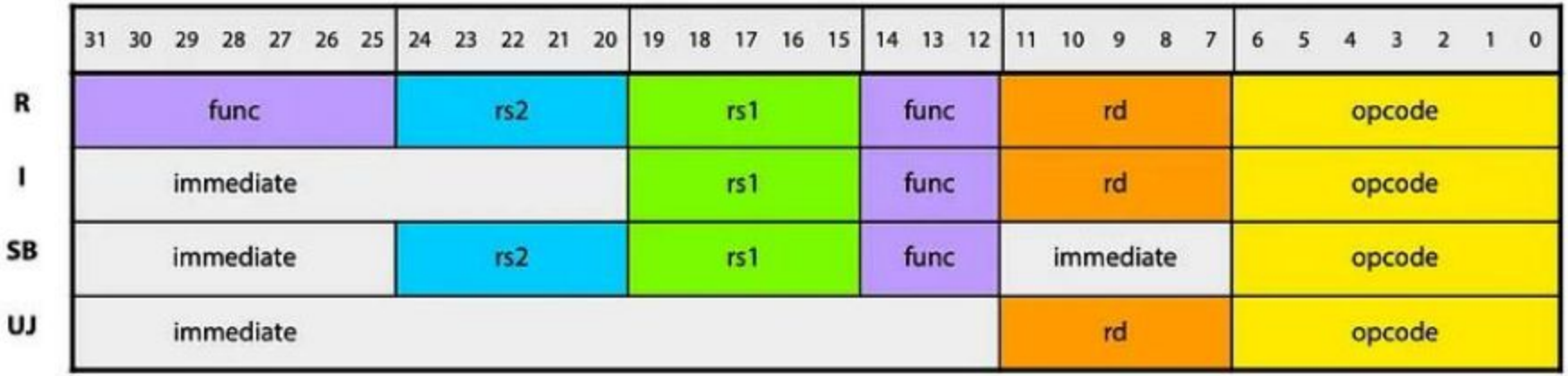
Logical Operations

Mnemonic	Instruction	Type	Description
`AND rd, rs1, rs2`	AND	R	`rd ← rs1 & rs2`
`OR rd, rs1, rs2`	OR	R	`rd ← rs1 | rs2`
`XOR rd, rs1, rs2`	XOR	R	`rd ← rs1 ^ rs2`
`ANDI rd, rs1, imm12`	AND immediate	I	`rd ← rs1 & imm12`
`ORI rd, rs1, imm12`	OR immediate	I	`rd ← rs1 | imm12`
`XORI rd, rs1, imm12`	XOR immediate	I	`rd ← rs1 ^ imm12`
`SLL rd, rs1, rs2`	Shift left logical	R	`rd ← rs1 << rs2`
`SRL rd, rs1, rs2`	Shift right logical	R	`rd ← rs1 >> rs2`
`SRA rd, rs1, rs2`	Shift right arithmetic	R	`rd ← rs1 >> rs2`
`SLLI rd, rs1, shamt`	Shift left logical immediate	I	`rd ← rs1 << shamt`
`SRLI rd, rs1, shamt`	Shift right logical imm.	I	`rd ← rs1 >> shamt`
`SRAI rd, rs1, shamt`	Shift right arithmetic immediate	I	`rd ← rs1 >> shamt`

Load / Store Operations

Mnemonic	Instruction	Type	Description
LD rd, imm12(rs1)	Load doubleword	I	rd ← mem[rs1 + imm12]
LW rd, imm12(rs1)	Load word	I	rd ← mem[rs1 + imm12]
LH rd, imm12(rs1)	Load halfword	I	rd ← mem[rs1 + imm12]
LB rd, imm12(rs1)	Load byte	I	rd ← mem[rs1 + imm12]
LWU rd, imm12(rs1)	Load word unsigned	I	rd ← mem[rs1 + imm12]
LHU rd, imm12(rs1)	Load halfword unsigned	I	rd ← mem[rs1 + imm12]
LBU rd, imm12(rs1)	Load byte unsigned	I	rd ← mem[rs1 + imm12]
SD rs2, imm12(rs1)	Store doubleword	S	rs2 → mem[rs1 + imm12]
SW rs2, imm12(rs1)	Store word	S	rs2(31:0) → mem[rs1 + imm12]
SH rs2, imm12(rs1)	Store halfword	S	rs2(15:0) → mem[rs1 + imm12]
SB rs2, imm12(rs1)	Store byte	S	rs2(7:0) → em[rs1 + imm12]

32-bit instruction format



```
.data
    num1:   .word 5      # Declare and initialize num1 with value 5
    num2:   .word 3      # Declare and initialize num2 with value 3
    result: .word 0      # Declare result variable with initial value 0

.text
    main:
        # Load num1 into register t0
        lw t0, num1

        # Load num2 into register t1
        lw t1, num2

        # Add num1 and num2 and store the result in register t2
        add t2, t0, t1

        # Store the result in memory
        sw t2, result

        # End program
        li a7, 10
        ecall
```

Simple Program in RISC

.data section:
Declares variables (num1, num2, result).
Initializes num1 = 5, num2 = 3.

.text section:
Contains main program logic, starting with the main label.

Load instructions:
lw (load word) loads num1 into register t0.
lw loads num2 into register t1.

Arithmetic operation:
add adds t0 and t1, storing the result in t2.

Store instruction:
sw (store word) stores t2 (result) in memory.

Program termination:
li (load immediate) loads system call number for termination into a7.
ecall makes the system call to end the program.

KNOWLEDGE CHECK

In a typical RISC processor, why is the load-store architecture used?

A) To allow arithmetic operations directly on memory addresses

B) To simplify the instruction set by separating memory access from computation

C) To enable variable-length instructions for better code density

D) To eliminate the need for CPU registers

KNOWLEDGE CHECK EXPLAINED...

B is correct because RISC's load-store architecture strictly separates memory access (load/store instructions) from computation (ALU operations on registers). This design simplifies pipelining (fixed-length instructions), reduces hardware complexity, & aligns with RISC’s philosophy of simple, single-cycle instructions.

A: Incorrect because RISC forbids arithmetic ops directly on memory (unlike CISC). All ALU operations work only on registers.
C: Incorrect because RISC uses fixed-length instructions, sacrificing code density for decode simplicity.
D: Incorrect because registers are critical in RISC (load-store relies on them). More registers = fewer slow memory accesses.

5. PARALLEL ARCHITECTURES

Parallel computing architecture involves the simultaneous execution of multiple computational tasks to enhance performance and efficiency.
In parallel computing, the architecture comprises essential components such as processors, memory hierarchy, interconnects, and software stack. These components work together to facilitate efficient communication, data processing, and task coordination across multiple processing units.

Parallel computing CPUs

Multi-core CPUs: These CPUs feature multiple processing cores integrated onto a single chip, allowing parallel execution of tasks. Each core can independently execute instructions, enabling higher performance and efficiency in multi-threaded applications.

Multi-threaded CPUs: Multi-threaded CPUs support the simultaneous execution of multiple threads within each core. This feature enhances throughput and responsiveness by overlapping the execution of multiple tasks, particularly in applications with parallelizable workloads.

Parallel Computing GPUs

Stream processors: GPUs consist of numerous stream processors, also known as shader cores, responsible for executing computational tasks in parallel. These processors are optimized for data-parallel operations and are particularly well-suited for graphics rendering, scientific computing, and machine learning tasks.

CUDA cores: CUDA (Compute Unified Device Architecture) cores are specialized processing units found in NVIDIA GPUs. These cores are designed to execute parallel computing tasks programmed using the CUDA parallel computing platform and application programming interface (API). CUDA cores offer high throughput and efficiency for parallel processing workloads.

Parallel Computing APUs (Accelerated Processing Units)

CPU cores: Accelerated Processing Units (APUs) integrate both CPU and GPU cores on a single chip. The CPU cores within APUs are responsible for general-purpose computing tasks, such as executing application code, handling system operations, and managing memory.

GPU cores: Alongside CPU cores, APUs also include GPU cores optimized for parallel computation and graphics processing. These GPU cores provide accelerated performance for tasks such as image rendering, video decoding, and parallel computing workloads.

Types of Parallel Architectures

SIMD (Single Instruction, Multiple Data)

- Computers that use the Single Instruction, Multiple Data (SIMD) architecture have multiple processors that carry out identical instructions. However, each processor supplies the instructions with its unique collection of data. SIMD computers apply the same algorithm to several data sets. The SIMD architecture has numerous processing components.
- All of these components fall under the supervision of a single control unit. While processing numerous pieces of data, each processor receives the same instruction from the control unit. Multiple modules included in the shared subsystem aid in simultaneous communication with every CPU. This is further separated into organizations that use bit-slice and word-slice modes.

Multiple Instruction, Single Data (MISD)

- Multiple processors are standard in computers that use the Multiple Instruction, Single Data (MISD) instruction set. While using several algorithms, all processors share the same input data. MISD computers can simultaneously perform many operations on the same batch of data. As expected, the number of operations is impacted by the number of processors available.
- The MISD structure consists of many processing units, each operating under its instructions and over a comparable data flow. One processor’s output becomes the input for the following processor. This organization’s debut garnered little notice and wasn’t used in architecture.

MIMD (Multiple Instruction, Multiple Data)

- Multiple Instruction, Multiple Data, or MIMD, computers are characterized by the presence of multiple processors, each capable of independently accepting its instruction stream. These kinds of computers have many processors. Additionally, each CPU draws data from a different data stream. A MIMD computer is capable of running many tasks simultaneously.
- Although MIMD computers are more adaptable than SIMD or MIMD computers, developing the sophisticated algorithms that power these machines is more challenging. Since all memory flows are changed from the shared data area transmitted by all processors, a MIMD computer organization incorporates interactions between the multiprocessors.
- The multiple SISD operation is equivalent to a collection of separate SISD systems if the many data streams come from various shared memories.

Single Program, Multiple Data (SPMD)

- SPMD systems, which stand for Single Program, Multiple Data, are a subset of MIMD. Although an SPMD computer is constructed similarly to a MIMD, each of its processors is responsible for carrying out the same instructions. SPMD is a message passing programming used in distributed memory computer systems. A group of separate computers, collectively called nodes, make up a distributed memory computer.
- Each node launches its application and uses send/receive routines to send and receive messages when interacting with other nodes. Systems can also use messages to provide barrier synchronization. It is possible to transfer the messages via a wide range of communication techniques, such as transmission control protocol (TCP/IP) over Ethernet and specialized high-speed interconnects, such as Supercomputer Interconnect and Myrinet.

SMP (Symmetric Multiprocessing)

- Symmetric Multiprocessing (SMP) is built around multiple identical processors sharing the same physical memory and managed by a single operating system instance. Each CPU can execute any task, including the operating system kernel, with equal priority.
- Data written by one processor is immediately accessible to all others, thanks to the shared memory model. To maintain accuracy, cache coherency protocols ensure that updates in one CPU’s cache are reflected across all caches and main memory.
- Processors are interconnected via a system bus, crossbar switch, or on-chip mesh, which acts as the communication highway. This setup eliminates the need for explicit messaging between CPUs, allowing seamless task scheduling, dynamic load balancing, and efficient parallel processing across all cores.

Clusters

- Cluster computing is a type of computing where multiple computers are connected so they work together as a single system. The term “cluster” refers to the network of linked computer systems programmed to perform the same task.
- Cluster computing architectures consist of a group of interconnected, individual computers working together as a single machine. Each computing resource is linked via a high-speed connection—such as a LAN—and referred to in the architecture of the system as a single node. Each node has an OS, memory and input and output (I/O) functions.
- There are two types of cluster architectures, open or closed. In an open cluster, each computer has its own IP address. In a closed cluster, each node is hidden behind a gateway node. Because the gateway node controls access to the other nodes and IP addresses can be found on the internet, closed clusters are less of a security risk than open clusters.

Self-Learning Activity

Define the following terms.

1. Bit-level parallelism
2. Instruction-level parallelism
3. Task parallelism
4. Superword-level parallelism

6. CHALLENGES IN PARALLELISM

Parallel computing offers high performance, but several challenges must be addressed for efficient execution.

Amdahl's Law

Amdahl's Law tells us that adding more processing units does not always yield proportional speed gains. This is a significant constraint for supercomputing, as it implies that tasks with even a small sequential component will face diminishing returns when scaling up parallel execution.

S(N) = 1/((1 - P) + P/N)

Where:

- S(N) is the speedup achieved using N processors.
- P is the proportion of the task that can be parallelized.
- 1 - P is the portion of the task that is inherently sequential and cannot be parallelized.

Even if a significant part of the task can be parallelized, the sequential portion ultimately dictates the maximum achievable speedup. As the number of processors (N) increases, the speedup plateaus, meaning that after a certain point, adding more processors provides little or no benefit due to the non-parallelizable segment of the computation. https://www.youtube.com/watch?v=tEmJvzXQeXU

Load Balancing

- Load balancing is the process of distributing network traffic efficiently among multiple servers to optimize application availability and ensure a positive end-user experience.
- Load balancing can be implemented in a couple of ways. Hardware load balancers are physical appliances that are installed and maintained on premises. Software load balancers are applications installed on privately-owned servers, or delivered as a managed cloud service (cloud load balancing).
- In either case, load balancers work by mediating incoming client requests in real time and determining which backend servers are best able to process those requests. In order to prevent a single server from becoming overloaded, the load balancer routes requests to any number of available servers on premises or hosted in server farms or cloud data centers.

Race Conditions and Synchronization

A race condition happens when the correctness of a program depends on the timing of parallel execution. To prevent this, synchronization mechanisms such as mutexes (mutual exclusion locks) or semaphores are used to control access to shared data.

However, excessive synchronization can lead to performance bottlenecks, as too many locks can serialize execution, reducing the benefits of parallelism. Careful optimization of synchronization mechanisms is critical for efficient supercomputing applications.

Pipeline Stalls

In pipeline parallelism, tasks are broken into sequential stages, where each stage must wait for the previous stage to finish. A pipeline stall occurs when a stage cannot proceed because it is waiting for data or resources from the previous stage, reducing efficiency.

- Data Dependencies – A later stage depends on the output of an earlier stage and must wait for completion.
- Resource Contention – Multiple stages compete for limited system resources, leading to delays.
- Branching and Control Flow – When execution flow depends on conditions, the pipeline may pause while waiting for decisions.

Granularity

Granularity (Grain Size) is a measure of the amount of work performed by a task.

- Fine-grained parallelism: Program is broken down into small tasks and assigned to to many processors. Work is evenly distributed.
- Coarse-Grained parallelism: Program is split into larger tasks. Most computations are performed sequentially within the processors. Difficult to load balance.

Fine-grained is more suitable for shared memory architectures, while coarse-grained is more suited for distributed systems.

7. USE CASES OF PARALLEL COMPUTING

Smartphones

Many smartphones rely on parallel processing to accomplish their tasks faster and more efficiently. For example, the iPhone 14 has a 6-core CPU and a 5-core GPU that power its market-leading ability to run 17 trillion tasks per second, an unthinkable level of performance using serial computing.

Supercomputers

Supercomputers rely so heavily on parallel computing that they are often called parallel computers. The American Summit supercomputer, for example, processes 200 quadrillion operations per second to help humans better understand physics and the natural environment.2

Blockchains

Blockchain technology, the technology that underpins cryptocurrencies, voting machines, healthcare and many other advanced applications of digital tech, relies on parallel computing to connect multiple computers to validate transactions and inputs. Parallel computing enables transactions in a blockchain to be processed simultaneously instead of one at a time, increasing throughput and making it highly scalable and efficient.

Laptop computers

Today’s most powerful laptops—MacBooks, ChromeBooks and ThinkPads—use chips with multiple processing cores, a foundation of parallelism. Examples of multicore processors include the Intel Core i5 and the HP Z8, which allow users to edit video in real-time, run 3D graphics and perform other complex, resource-intensive tasks.

Internet of Things

The Internet of Things (IoT) relies on data that is collected from sensors, which are connected to the internet. Once that data has been collected, parallel computing is required to analyze it for insights and help complex systems, such as power plants, dams and traffic systems, function. Traditional serial computing can’t sift through data fast enough for IoT to work, making parallel computing crucial to the advancement of IoT technology.

Artificial intelligence and machine learning

Parallel computing plays a critical role in the training of ML models for AI applications, such as facial recognition and natural language processing (NLP). By performing operations simultaneously, parallel computing significantly reduces the time that it takes to train ML models accurately on data.

THANK YOU

19.05.2026